AI-BASED PRE-HOSPITAL EMERGENCY ALERT AND SMART HOSPITAL RESOURCE ALLOCATION SYSTEM

(EmergencyAI)

A PROJECT REPORT

Submitted by

2116240701451 – SAHANA A

2116240701453 – SAHANA P



October 2026

BONAFIDE CERTIFICATE

Certified that this project report entitled “AI-Based Pre-Hospital Emergency Alert and Smart Hospital Resource Allocation System” is the bonafide work of **SAHANA A, SAHANA P** bearing register number(s) **240701451,240701453** who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____.

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	3
1	INTRODUCTION	5
2	LITERATURE REVIEW	7
3	PROPOSED SYSTEM	9
3.1	Architecture Diagram	10
4	MODULES DESCRIPTION	11
4.1	Emergency Registration Module	12
4.2	Rule-Based Severity Analysis Module	13
4.3	Ambulance Management Module	14
4.4	Hospital Resource and Allocation Module	14
4.5	Dashboard and Alert Module	15
5	IMPLEMENTATION AND RESULTS	15
5.1	Experimental Setup	16
5.2	Results	16
6	CONCLUSION AND FUTURE WORK	17
7	REFERENCES	17

Note: Fill in the page numbers after final editing and pagination.

ABSTRACT

EmergencyAI is a software-based prototype designed to support pre-hospital emergency coordination and smart hospital resource allocation. During an emergency, coordinators may need to review patient information, understand the seriousness of the case, identify an available ambulance, and determine which hospital may have the resources required. EmergencyAI brings these activities together through a centralized dashboard. The system accepts patient and incident details such as age, emergency type, location, vital signs, symptoms, injury information, loss of consciousness, and required specialist. Its explainable rule-based decision engine applies predefined conditions to generate a severity category, a score, risk factors, and a recommended action. The system then uses the available ambulance and hospital information to support ambulance selection and hospital recommendation. Hospital resource information may include ICU beds, emergency beds, ventilators, specialist availability, and occupancy. The application is implemented using React and Vite for the frontend, Python FastAPI for the backend, and SQLite with SQLAlchemy for database operations. The frontend communicates with backend endpoints through REST APIs. The dashboard includes emergency information, ambulance and hospital data, summary statistics, and alerts. The current prototype uses rule-based logic; it does not implement machine learning or deep learning, and its outputs have not been clinically validated. Fictional data is used for demonstration. EmergencyAI is intended to demonstrate how emergency information and resource coordination

can be organized in one workflow, not to replace qualified medical professionals or real emergency dispatch services.

1. INTRODUCTION

1.1 Background

Emergency response involves several connected activities. Information about the patient and incident must be collected, the urgency of the case must be assessed, an ambulance may need to be assigned, and a suitable hospital must be identified. When these details are spread across separate tools or handled manually, it can be difficult for a coordinator to maintain a unified view of the incident and available resources.

Digital decision-support applications can help organize emergency details and present relevant information in a centralized interface. EmergencyAI is a prototype developed to demonstrate this approach by combining emergency registration, rule-based severity analysis, ambulance management, hospital resource matching, and alerts.

1.2 Problem Statement

Emergency coordinators need a clear and organized way to view emergency case details alongside ambulance availability and hospital resources. Separate information sources can make it harder to compare requirements and available resources quickly. The project addresses this coordination problem by developing a centralized dashboard and a transparent rule-based workflow.

1.3 Objectives

- To provide a form for recording patient and emergency information.
- To apply predefined rules to produce an explainable severity category and associated risk factors.

- To display ambulance availability and support ambulance assignment and status updates.
- To maintain hospital resource information and recommend a hospital based on case requirements and available resources.
- To present emergency statistics and alerts through a centralized dashboard.
- To demonstrate communication between a React frontend and a FastAPI backend using REST APIs.

1.4 Scope

The scope of the current implementation is a local software prototype using fictional demonstration data. It demonstrates information entry, rule-based analysis, resource display, and coordination workflows. Real ambulance GPS, live hospital information systems, validated clinical protocols, and production dispatch integrations are outside the current scope.

1.5 Limitations

The severity rules are predefined and have not been validated for clinical use. The system does not use a trained machine-learning or deep-learning model. Recommendations depend on the correctness and completeness of the data stored in the prototype. Therefore, outputs must not be treated as medical diagnosis or used as a substitute for professional judgement.

2. LITERATURE REVIEW

Emergency response and healthcare information systems commonly address different parts of the coordination process. The following review summarizes general categories of existing approaches and the design considerations that informed EmergencyAI. It is a category-based review rather than a claim that a particular published study was experimentally reproduced.

2.1 Traditional Emergency Coordination

Traditional workflows may depend on phone calls, manual records, and communication between dispatchers, ambulance teams, and hospitals. These approaches can work in established operational settings, but information may be distributed across different people or systems. A centralized dashboard can improve visibility by bringing case details and resource status into one interface.

2.2 Ambulance Tracking and Dispatch Systems

Ambulance management systems typically focus on vehicle status, location, dispatch, and estimated arrival time. These functions are important for response coordination. However, ambulance status alone does not show whether a destination hospital has the resources required by a particular patient. EmergencyAI therefore displays ambulance information alongside hospital resource information.

2.3 Hospital Information and Resource Management Systems

Hospital information systems maintain operational information such as beds, departments, and clinical services. These systems are generally designed around hospital operations. For pre-hospital coordination, a coordinator also needs the emergency details and transport status. EmergencyAI models a simplified resource view that can be considered together with the incoming case.

2.4 Rule-Based Decision Support

A rule-based system applies explicit conditions to input data and produces outputs according to those conditions. Its reasoning can be inspected because the rules are predefined. This makes it useful for a transparent prototype where the goal is to show how input factors influence an output. Its limitations include dependence on the quality of the rules and inability to learn automatically from new data.

2.5 Research Gap and Proposed Contribution

The project focuses on integrating emergency intake, explainable severity scoring, ambulance status, hospital resource matching, and alerts in one workflow. Its contribution is a working software prototype demonstrating this integration. It does not claim that all existing systems lack these features or that the prototype has demonstrated improved clinical outcomes.

2.6 Summary

The review suggests that a useful prototype should provide centralized visibility, transparent decision logic, and a clear relationship between emergency requirements and available resources. These principles guide the design of EmergencyAI.

3. PROPOSED SYSTEM

EmergencyAI is a web-based decision-support prototype. An emergency coordinator enters case information through the frontend. The frontend sends the information to the FastAPI backend, where the application processes the request, applies the rule-based severity logic, and consults stored ambulance and hospital information. The backend returns a response that the dashboard can display.

3.1 Architecture Diagram

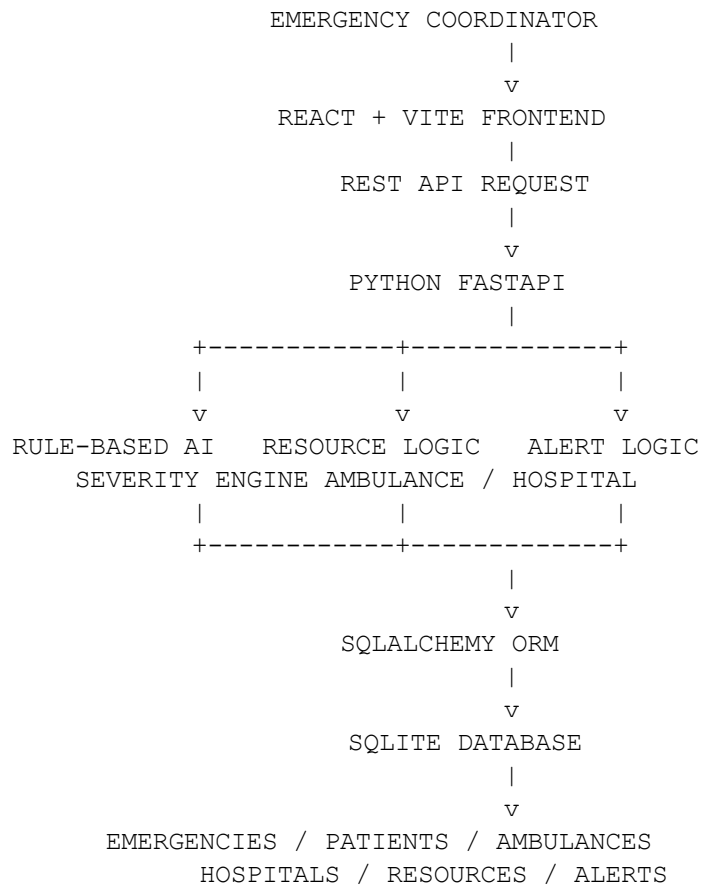


Figure 3.1: High-level architecture of EmergencyAI.

3.2 System Workflow

1. The coordinator enters the patient's details, emergency type, location, vital signs, symptoms, and requirements.
2. The backend validates the request according to the application's input schema.
3. The rule-based engine evaluates the submitted values and returns a severity level, score, risk factors, and recommended action.
4. The application checks the available ambulance information and prepares an assignment when a suitable ambulance is available.

5. The hospital recommendation logic evaluates the case requirements against the hospital resource information stored in the database.
6. The system records the emergency and related alert information, then returns a response to the frontend.
7. The dashboard displays the case, severity, ambulance status, hospital recommendation, and alerts.

3.3 Functional Requirements

- The system shall display emergency, ambulance, hospital, dashboard-statistics, and alert information.
- The system shall accept emergency information through a structured form.
- The system shall apply predefined rules and return an explainable severity result.
- The system shall display ambulance status and hospital resource details.
- The system shall provide hospital recommendation and alert functionality supported by the backend.
- The system shall store application records in a local SQLite database.

3.4 Non-Functional Requirements

The interface should be understandable, the API should return structured responses, and the system should keep its modules separated so that the prototype can be extended. Data accuracy, access control, privacy, reliability, and clinical safety would require additional work before any real-world deployment.

4. MODULES DESCRIPTION

4.1 Emergency Registration Module

This module collects the details required to describe an emergency case. The fields include patient name, age, gender, emergency type, location, heart rate, SpO_2, blood pressure, respiratory rate, temperature, symptoms, injury type, loss of consciousness, and required specialist. The form sends the information to the

backend through the emergency creation endpoint. The backend creates the related patient and emergency records and returns the result for display.

The module is intended to standardize the information captured for each case. Input validation should ensure that required fields are present and values use the expected format. Demonstration cases must use fictional data.

4.2 Rule-Based Severity Analysis Module

This module evaluates the submitted emergency information using predefined rules. It considers factors such as oxygen saturation, heart rate, blood pressure, respiratory rate, temperature, symptoms, injury type, and loss of consciousness. The logic produces a severity category, a score, a list of risk factors, a confidence field if supplied by the implemented logic, and a recommended action.

The module is explainable because the output can include the factors that triggered the rules. It is not a trained model and does not learn from data. The thresholds and outcomes must be reviewed by qualified medical professionals before any clinical use; the current prototype is for demonstration only.

4.3 Ambulance Management Module

The ambulance module displays ambulance records and their status. The prototype supports statuses such as AVAILABLE and EN ROUTE and stores fields such as ambulance identifier, assigned patient, destination, and estimated time of arrival. When an emergency is processed, the backend attempts to identify an available ambulance using its implemented selection logic. If one is selected, the corresponding status and assignment details are updated.

The current implementation should not be described as live GPS tracking unless a real location feed is added. Location and estimated response values in a demo may be simulated or based on stored data.

4.4 Hospital Resource and Allocation Module

This module stores and displays hospital information. Resource fields can include ICU beds, emergency beds, ventilators, occupancy, and specialist availability. The recommendation logic compares the requirements of the emergency with the available hospital information and returns a candidate hospital when one can be identified.

The recommendation is a resource-matching result based on the data available in the prototype. It does not confirm real-time capacity or guarantee clinical suitability. A production system would require secure integration with verified hospital systems and regular updates.

4.5 Dashboard and Alert Module

The dashboard provides a consolidated view of active emergencies, ambulance availability, hospitals, resource counts, and recent alerts. It requests information from backend REST endpoints and displays the returned data in dashboard cards, lists, and tables. Alert records can describe events such as a new emergency, ambulance dispatch, or hospital notification.

This module is intended to help a coordinator see the current state of the prototype without navigating multiple separate screens. The displayed statistics depend on the database records and logic currently implemented.

4.6 Database and API Module

The backend uses FastAPI to expose REST endpoints. SQLAlchemy maps Python models to database tables, and SQLite stores the prototype's records locally. The application includes endpoints for dashboard statistics, emergencies, ambulances, hospitals, allocation, analytics, and alerts. Separating API routes, database models, schemas, and service logic supports maintenance and future extension.

5. IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

The prototype is developed and tested locally on a computer. The frontend uses React and Vite. The backend uses Python and FastAPI. SQLAlchemy provides database access to SQLite. A modern web browser is used to access the dashboard, and FastAPI's interactive API documentation can be opened at the local /docs route when the backend is running.

5.1.1 Software Requirements

- Operating system: macOS or another compatible development environment.
- Frontend runtime and package manager: Node.js and npm.
- Frontend framework/build tool: React and Vite.
- Backend language/framework: Python and FastAPI.
- Database and ORM: SQLite and SQLAlchemy.
- Browser: a modern browser such as Chrome, Firefox, Safari, or Edge.

5.1.2 Running the Prototype

Start the backend from the backend directory after activating the project's Python environment and installing its requirements. Run the FastAPI application with Uvicorn, for example using the command `uvicorn main:app --reload`. Start the frontend in a separate terminal from the frontend directory using `npm install` when dependencies are not installed, followed by `npm run dev`. Open the local address printed by Vite in a browser. Keep both terminal processes running while testing the application.

5.1.3 Test Procedure

8. Start the backend and confirm that the application startup completes without errors.
9. Open the frontend and check whether the dashboard loads its data.
10. Check the ambulance, hospital, active emergency, dashboard statistics, and alert endpoints.

11. Enter fictional emergency details and submit the form if the form is enabled in the current build.
12. Inspect the returned severity, risk factors, ambulance selection, hospital recommendation, and alert information.
13. Record any failed requests or unexpected results and correct them before the final demonstration.

5.2 Results

During local testing, the backend successfully started and the following dashboard data endpoints returned HTTP 200 OK: `/api/ambulances`, `/api/hospitals`, `/api/emergencies/active`, `/api/dashboard/stats`, and `/api/alerts`. An HTTP 200 response indicates that the corresponding request was processed successfully. This confirms that the listed read endpoints were reachable during that test; it does not by itself prove that every form submission or allocation workflow is error-free.

The prototype is designed to display emergency information, ambulance and hospital data, dashboard statistics, and alerts. The emergency creation workflow is intended to apply the rule-based severity logic, attempt ambulance selection, recommend a hospital, and return the result to the frontend. Before submitting the final report, the team should test this workflow end-to-end and add genuine screenshots from the running application.

5.2.1 Sample Demonstration Scenario

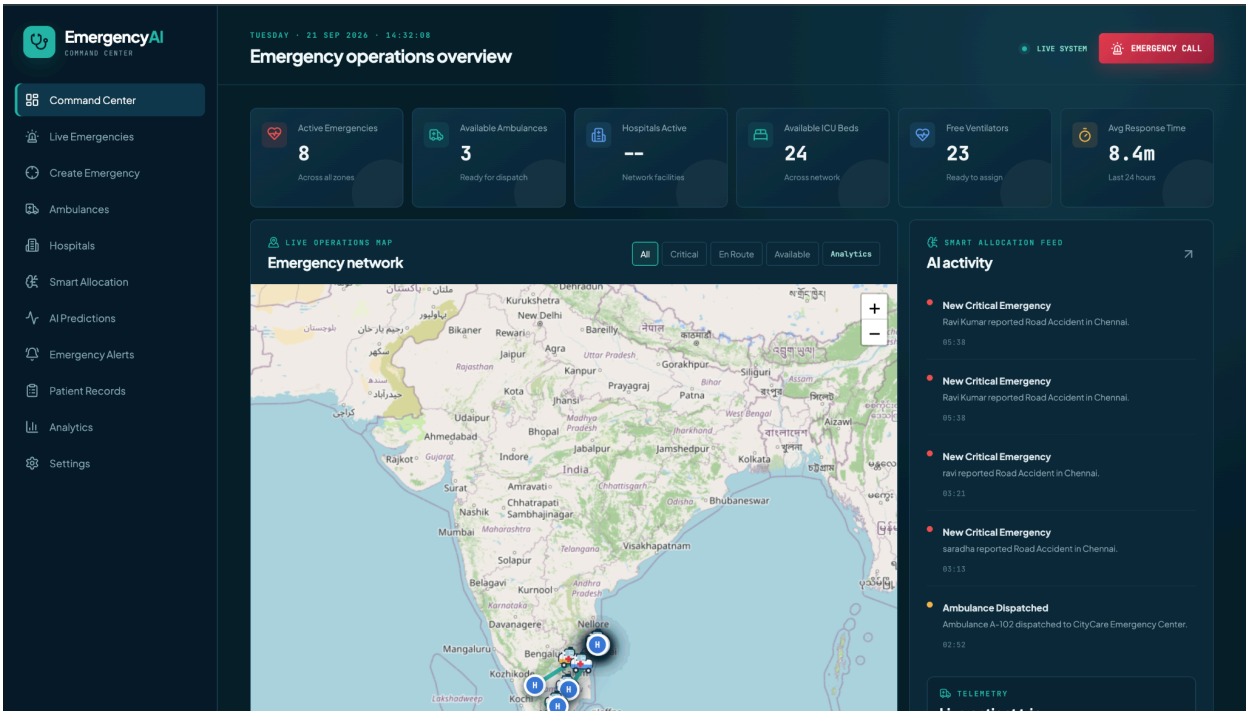
The following is a fictional scenario for explaining the intended workflow, not a real patient record or clinically validated output.

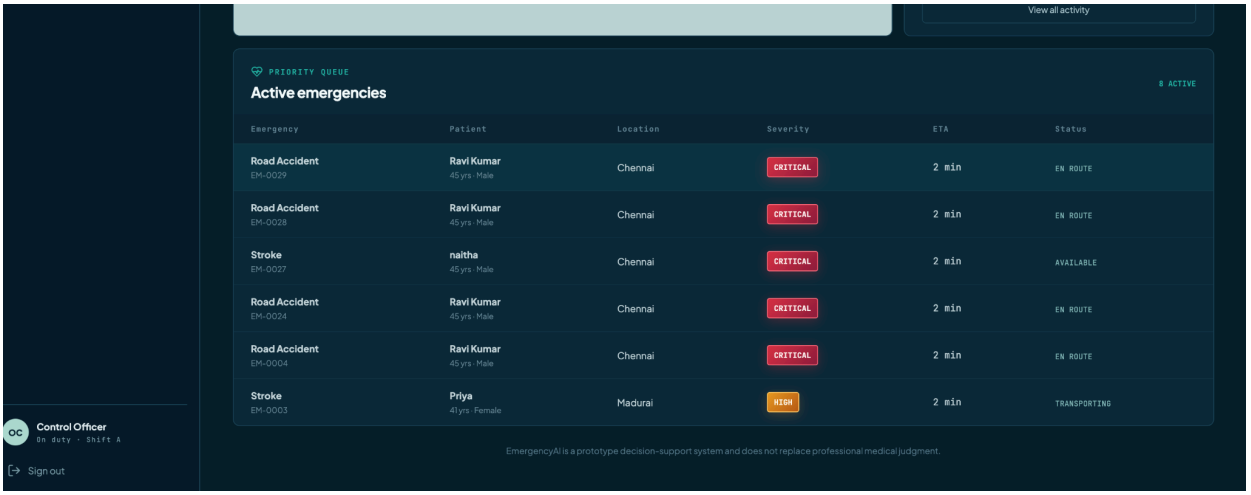
Patient name	Ravi Kumar (fictional)
Age	45
Emergency type	Road accident
Location	Chennai (demo location)
Example input	SpO_2 85%, heart rate 120, blood pressure 90/60, respiratory rate 28
Injury details	Head injury; loss of consciousness marked Yes
Required specialist	Neurosurgeon
Expected prototype display	Severity result and risk factors from the configured rules; ambulance selection if available;

	hospital recommendation if suitable resource data exists; alert record

The actual severity category and allocation result must be copied from the running application after testing. Do not state that an ambulance or hospital was successfully allocated unless the demonstration confirms it.

5.2.2 Screenshots





5.2.3 Result Discussion

The successful responses from the dashboard endpoints show that the frontend can retrieve core information from the backend during the observed test. The design also provides a structure for integrating severity analysis and resource matching.

Further testing is required for invalid inputs, database failures, duplicate patient identifiers, no available ambulance, no suitable hospital, and concurrent requests. A production version would additionally require authentication, authorization, audit logging, data protection, secure hosting, and validation of all decision rules.

6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

EmergencyAI demonstrates a centralized software workflow for pre-hospital emergency coordination and hospital resource allocation. It combines emergency registration, explainable rule-based severity analysis, ambulance management, hospital resource matching, and alerts through a web dashboard. React and Vite provide the user interface, FastAPI exposes backend endpoints, and SQLite with SQLAlchemy stores prototype data. Local testing confirmed successful responses from the dashboard's ambulance, hospital, active-emergency, statistics, and alert endpoints.

The project is a prototype rather than a clinically validated system. Its current decision engine uses predefined rules and does not use machine learning or deep learning. The results depend on the quality of the configured rules and the demonstration data. The system should not be used for real patient triage or dispatch without appropriate clinical, technical, security, and operational validation.

6.2 Future Work

- Review and validate emergency rules with qualified medical professionals before any clinical use.
- Integrate secure, verified hospital information systems for up-to-date bed, ventilator, and specialist availability.
- Add GPS-based ambulance tracking and reliable travel-time estimation.
- Improve input validation, error handling, logging, authentication, and role-based access control.

- Add audit trails and privacy safeguards for sensitive patient information.
- Conduct systematic tests for edge cases, concurrent requests, and failures in external services.
- Deploy the application to a secure cloud environment and develop a mobile-friendly interface.
- Evaluate whether validated historical data could support future analytics or ML/DL research, subject to appropriate governance and clinical oversight.

7. REFERENCES

FastAPI (n.d.) ‘FastAPI Documentation’. Available at: https://fastapi.tiangolo.com/ (Accessed: 20 October 2026).

React (n.d.) ‘React Documentation’. Available at: https://react.dev/ (Accessed: 20 October 2026).

SQLAlchemy (n.d.) ‘SQLAlchemy Documentation’. Available at: https://docs.sqlalchemy.org/ (Accessed: 20 October 2026).

SQLite (n.d.) ‘SQLite Documentation’. Available at: https://www.sqlite.org/docs.html (Accessed: 20 October 2026).

Vite (n.d.) ‘Vite Documentation’. Available at: https://vite.dev/guide/ (Accessed: 20 October 2026).

World Health Organization (2019) ‘Emergency Care Systems Framework’. Available at: https://www.who.int/publications/i/item/WHO-EMC-2019.1 (Accessed: 20 October 2026).